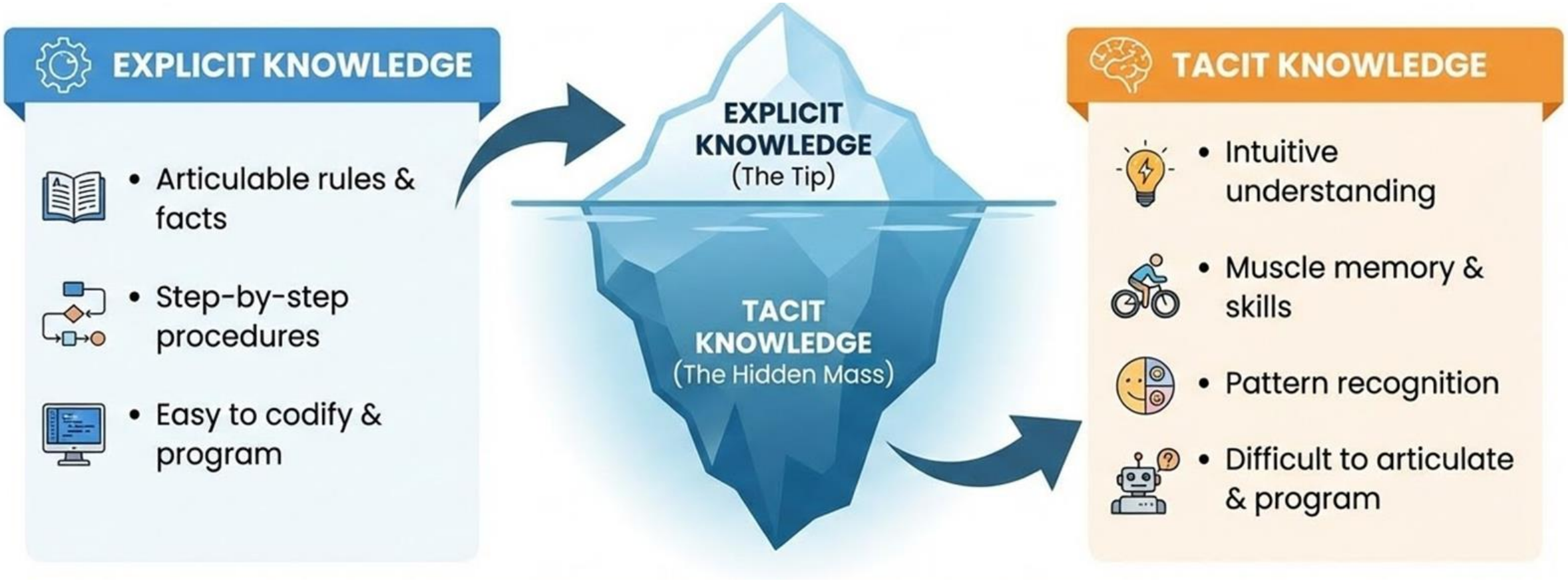
Applied AI

Text Vectorisation

Goals of this lecture…

- Understand why raw text must be converted to numerical vectors for machine learning.
- Understand the text vectorization process, including the three main approaches to tokenization.
- Contrast One-Hot Encoding with Dense Embeddings i.e. (Sparse hard-rules vs. Dense semantic relationships).

Recall – Polanyi's Paradox



Recall – Polanyi's Paradox

We cannot create if-then rules to classify an image of a cat.

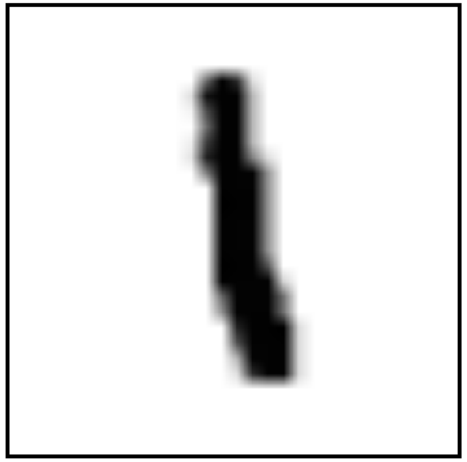
Even worse we can't just describe a cat to a model.

Models require numerical inputs to function.

Recall: Image classification

Last time we saw that we can create models ourselves to classify images using Keras.

Here, the images are fed into the model as grid or column of pixel vales



$$\simeq \begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & .6 & .8 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & .7 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & .7 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & .5 & 1 & .4 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & .4 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & .4 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & .7 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & .9 & 1 & .1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & .3 & 1 & .1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{bmatrix}$$

NLP: Handling Text

How could we create an AI model to translate from English to Spanish.

Early Approach: Use hand-crafted rules.

- Write down the ‘rule set for English’ and other languages.
- These rules and relations between them are defined by experts
- This approach was flawed.

NLP: Handling Text

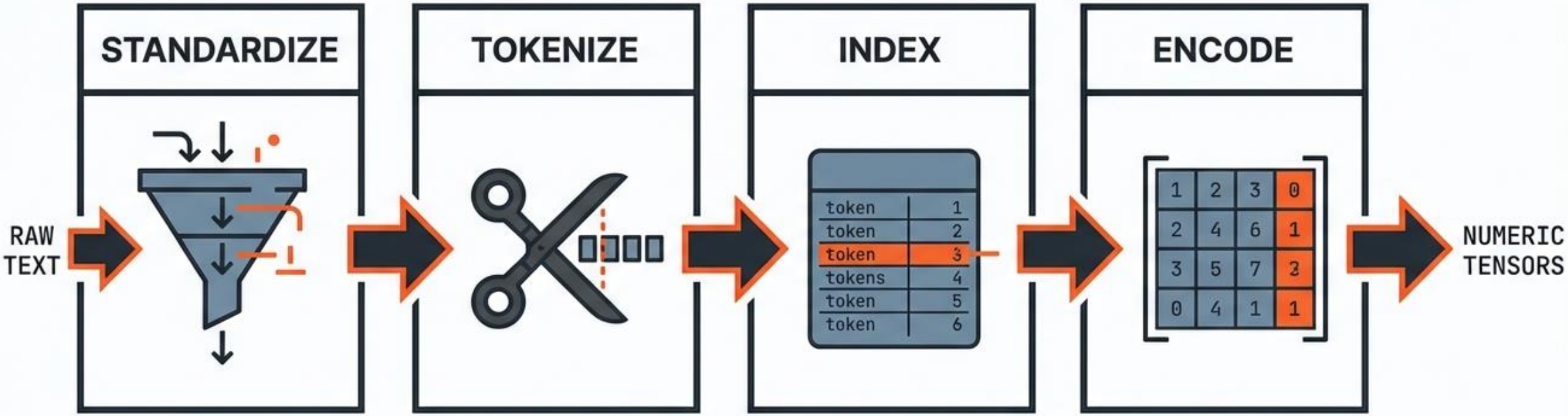
To use Machine Learning models and, in particular, Deep Learning models we need to first deal with the issue of text.

- Models, cannot process the text directly.
- We need numerical inputs.

So, *how do we convert the text into numerical inputs?*

- This is the process (text vectorisation) we shall go through today.

TEXT VECTORIZATION PIPELINE



Cleans and normalizes text data.

Breaks text into smaller units (tokens).

Assigns unique IDs to each token.

Converts indexed tokens into numeric tensors.

TEXT VECTORIZATION: The process of converting strings into numeric tensors.

Standardisation

This is the process of normalising the text. Traditionally, this has involved:

- Stripping white spacing
- Strip capitalisations and punctuation
- Removing stop words (a, the, it, and,..)
- Stemming (“tired” → “tire” and “trophies” → “trophy”)

Standardisation

For example, consider these two sentences

“sunset came. i was staring at the Mexico sky. Isnt nature splendid??”

“Sunset came; I stared at the México sky. Isn’t nature splendid?”

If we remove capitals, punctuation and apply stemming these become essentially the same.

Standardisation

"sunset came. i was staring at the Mexico sky. Isnt nature splendid??"

Remove capitals:

"sunset came. i was staring at the mexico sky. isnt nature splendid??"

Remove punctuation:

"sunset came i was staring at the mexico sky isnt nature splendid"

Standardisation

Apply Stemming:

"sunset came i was [stared] at the mexico sky isnt nature splendid"

Doing the same steps to

"Sunset came; I stared at the México sky. Isn't nature splendid?"

Would result in essentially same result especially if stop words removed.

Historical Importance

Consider the following emails:

"WIN money now!!!"
"Win Money Now"
"win money now"

Without standardisation these are completely unique:

- WIN ≠ Win ≠ win etc
- "!!!" becomes its own token

The unique features for a module to process (vocabulary size) increases unnecessarily

Historical Importance

Problems: without standardisation the models would have:

- Sparse feature explosion
- Fewer counts per feature
- Potentially poor generalisation
- Higher memory + compute cost

Especially difficult for historic models which were count based.

Historical Importance

After standardisation (lowercase + remove punctuation):

"win money now"

Now the model:

- Sees consistent word frequencies
- Learns patterns with less data
- Uses fewer features

Historic count-based models would work much better.

Standardisation

Q: For each of the following think of some drawbacks:

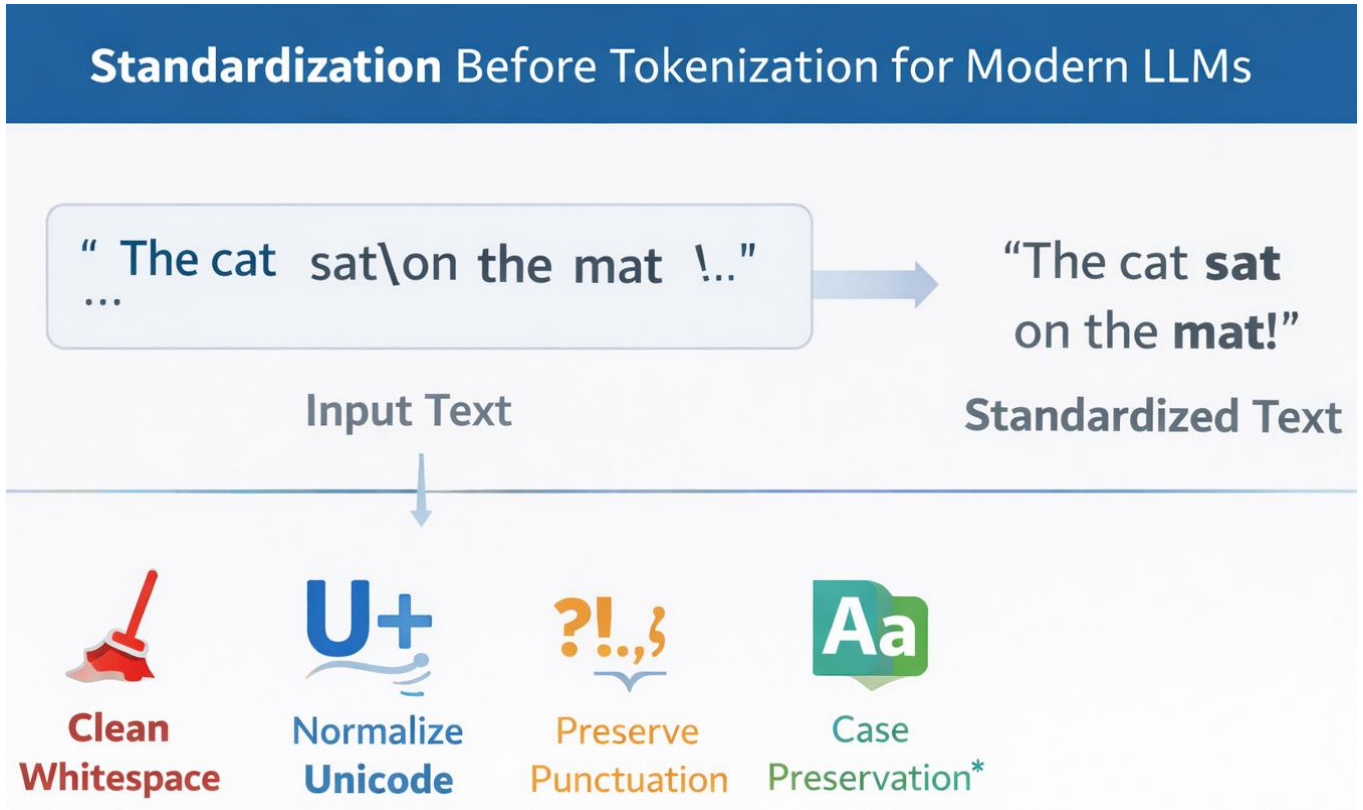
- Strip capitalisations and punctuation:

- Removing stop words:

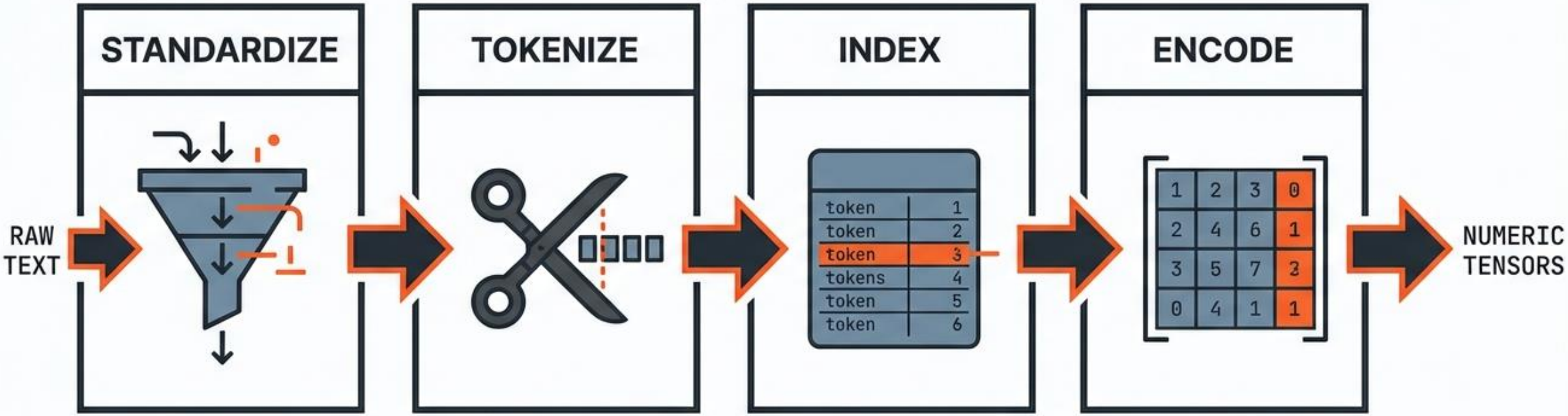
- Stemming:

Standardisation - Modern

In modern modelling (LLM) we usually only apply whitespace cleanup to the text.



TEXT VECTORIZATION PIPELINE



Cleans and normalizes text data.

Breaks text into smaller units (tokens).

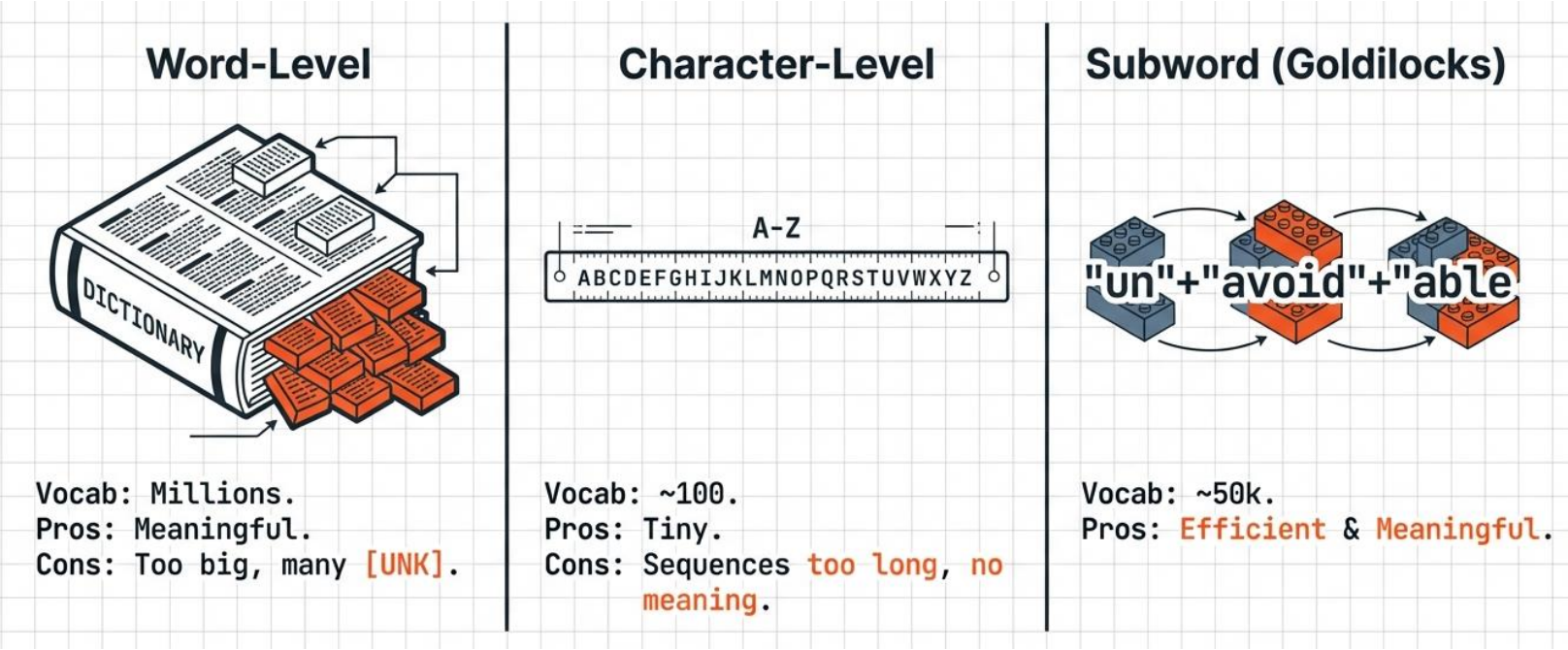
Assigns unique IDs to each token.

Converts indexed tokens into numeric tensors.

TEXT VECTORIZATION: The process of converting strings into numeric tensors.

Tokenisation

This is where we break (split) our string of text down into parts called tokens which are then used by the models.



Tokenisation

The *character-level* approach is probably the easiest to implement.

“sunset came i was staring at the mexico sky isnt nature splendid”

Splitting on characters would give:

[‘s’, ‘u’, ‘n’, ‘s’, ‘e’, ‘t’, ‘ ‘, ‘c’, ‘a’, ‘m’, ‘e’, ‘ ‘, ‘i’, ‘ ‘, ‘w’, ‘a’, ‘s’, ‘ ‘, ‘s’ ..]

Tokenisation

So, if this sentence was our entire training set then the set of unique tokens (vocabulary) would be

[‘a’, ‘c’, ‘d’, ‘e’, ‘g’, ‘h’, ‘i’, ‘k’, ‘l’, ‘m’, ‘n’, ‘p’, ‘r’, ‘s’, ‘t’, ‘u’, ‘w’, ‘x’, ‘y’, ‘ ‘, [UNK]]

Which is of size 21. In general,

- Small Vocabulary: Relatively small for character-level
- Flexible: Can combine to make general strings with less unknown [UNK] parts (tokens)
- Long Sequences: To represent strings takes longer seq of tokens

Indexing

Once, we have a vocabulary we then index the tokens i.e. associate with an integer:

[‘a’, ‘c’, ‘d’, ‘e’, ‘g’, ‘h’, ‘i’, ‘k’, ‘l’, ‘m’, ‘n’, ‘p’, ‘r’, ‘s’, ‘t’, ‘u’, ‘w’, ‘x’, ‘y’, ‘ ‘, [UNK]]

We can do this based on their position above i.e. ‘a’ -> 0, ‘d’ -> 2 etc. So,

“the cat sat on the mat”

becomes

14, 5, 3, 19, 1, 0, 14, 19, 13, 0, 14, 19, 20, 10, 19, 14, 5, 3, 9, 0, 14

Tokenisation

The *word-level* approach feels the most natural.

“sunset came i was staring at the mexico sky isnt nature splendid”

Splitting on into sentences would give:

[‘sunset’, ‘came’, ‘i’, ‘was’, ‘staring’, ‘at’, ‘the’, ‘mexico’, ‘sky’, ‘isnt’, ‘nature’, ‘splendid’]

Example

Let’s try to apply BPE to our sentence:

“sunset came i was staring at the mexico sky isnt nature splendid”

Tokenisation

With this one sentence our vocabulary is of size 13 as below:

[‘sunset’, ‘came’, ‘i’, ‘was’, ‘staring’, ‘at’, ‘the’, ‘mexico’, ‘sky’, ‘isnt’, ‘nature’, ‘splendid’, [UNK]]

In general,

- Larger Vocabulary: With larger training data all possible ‘words’ forms a large vocabulary.
- Less Flexible: [UNK] more common (e.g. spelling mistakes)
- Shorter sequences: Compact sequences as more information per-token.

Indexing

If we index this vocabulary in order as below:

[‘sunset’, ‘came’, ‘i’, ‘was’, ‘staring’, ‘at’, ‘the’, ‘mexico’, ‘sky’, ‘isnt’, ‘nature’, ‘splendid’, [UNK]]

Then the sentence,

“the cat sat on the mat”

becomes

6, 12, 12, 12, 6, 12

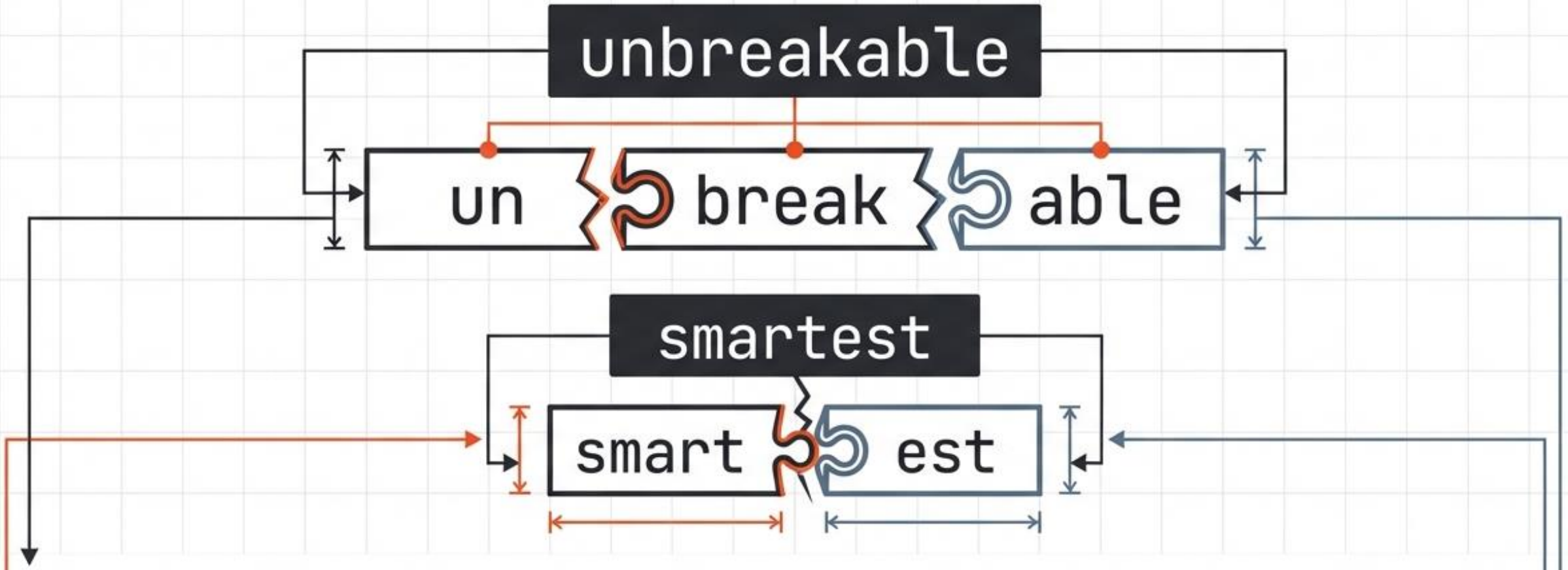
Exercise

Tokenisation - Subword

The problems we have seen with Character and Word level tokenisation has led to the rise of *subword tokenisation. Goal:*

With a reasonable small vocabulary we want to have shorter sequences but also encode a wide range of inputs

The tokens are learnt from the training data using frequency/statistical objectives.



- Fixes the [UNK] problem: unknown words are built from known parts.
- Learns morphology: 'un-' means negation, '-est' means superlative.
- The standard for modern LLMs (GPT, Llama).

Tokenisation - Subword

What does “learning” mean in the subword tokenisation?

Here we learn the ‘best’ *vocabulary of word pieces* and *the rules* for how to split text into those pieces.

Important: this is NOT the model we plan to use learning task.

It is a separate preprocessing stage that is usually done before model training.

Tokenisation - Subword

How is subword learning done?

Subword vocabularies are typically learned from a *large text corpus using frequency/statistical objectives.*

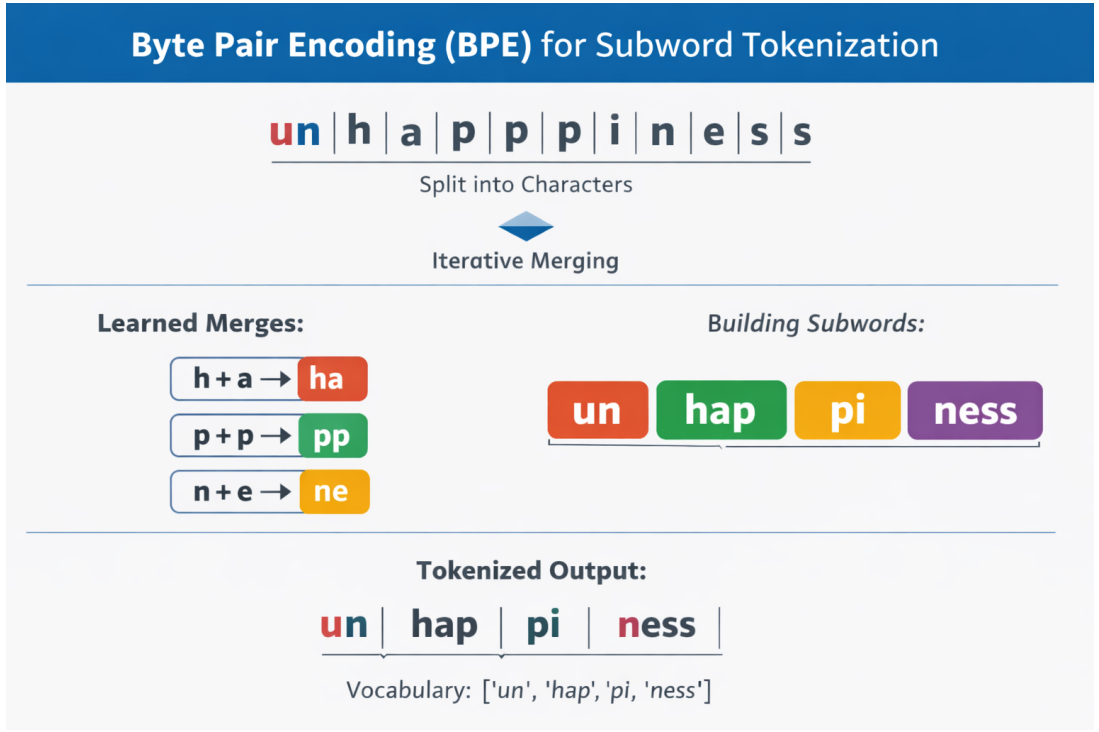
There are common algorithms which take different approaches to this such as Byte Pair Encoding (BPE), WordPiece or Unigram.

- We will focus on BPE.

BPE

- **Byte-Pair Encoding (BPE)** was originally developed in the 1990s as a lossless data compression technique.

- It is widely used by modern large language models, including the GPT family, because it elegantly handles punctuation, casing, and entirely newly invented words

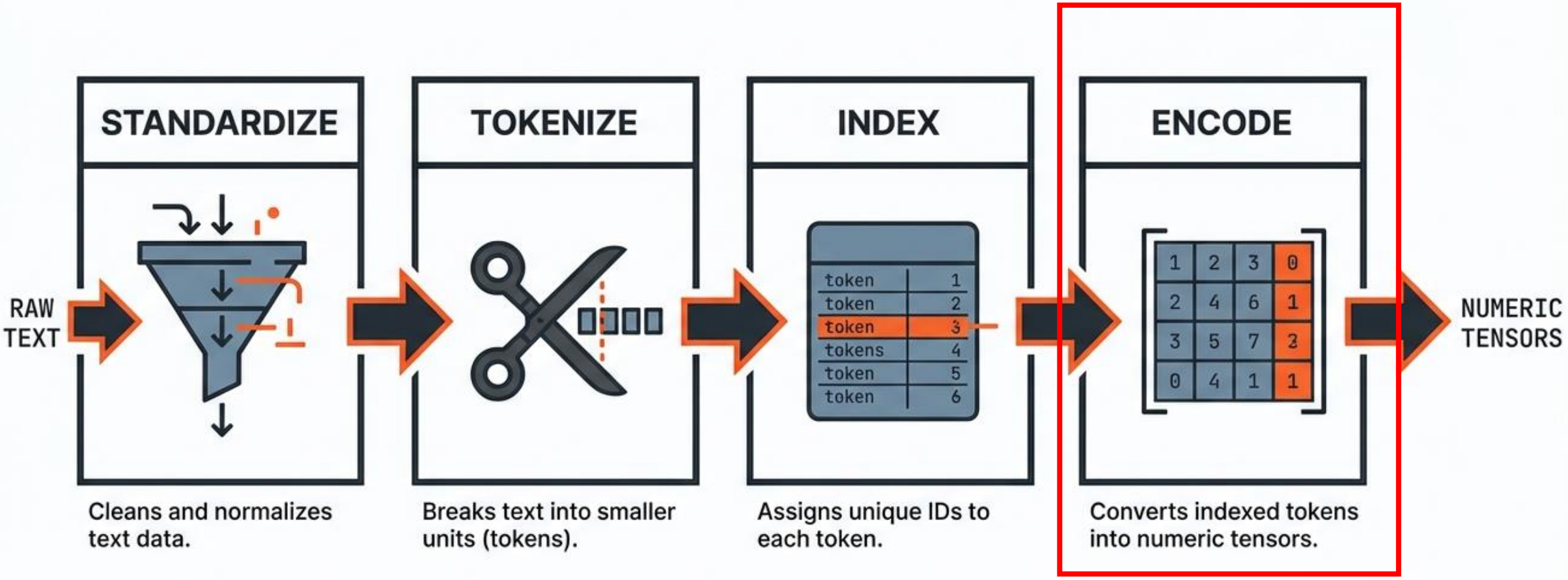


BPE

1. Break training data down to character-level. This forms the base vocabulary.
2. Learning (counting & merging):
   a. Look at all adjacent pairs of characters across text and count frequency.
   b. Merge the most common pair into new symbol which is added to vocabulary.
3. Build iteratively:
   a. Continue this counting and merging process iteratively. New symbols act as single unit which can be merged with characters or other symbols.

Exercise

TEXT VECTORIZATION PIPELINE



TEXT VECTORIZATION: The process of converting strings into numeric tensors.

Encoding

At this stage we have a way of:

- Standardising text
- Breaking that into tokens.
- Assign an index to each token to convert string of text into a sequence of numbers.

Encoding is then the process of turning these token IDs into vectors to be processed by our models.

Why Encode?

Think of encoding as giving every token its own *coordinate representation* so the model can:

- combine tokens in a sentence (sequence of vectors)
- compute “compatibility” between tokens (attention)
- gradually learn patterns by adjusting weights

Without vectors, the model has nothing sensible to *compute* on.

One-hot vs Embeddings

There are two main ways to encode a token ID.

1. One-hot encoding: a vector that *only identifies* the token (no meaning, just identity)

2. Embeddings: a vector that *can carry meaning* because it's learned and dense

We well now contrast these two approaches.

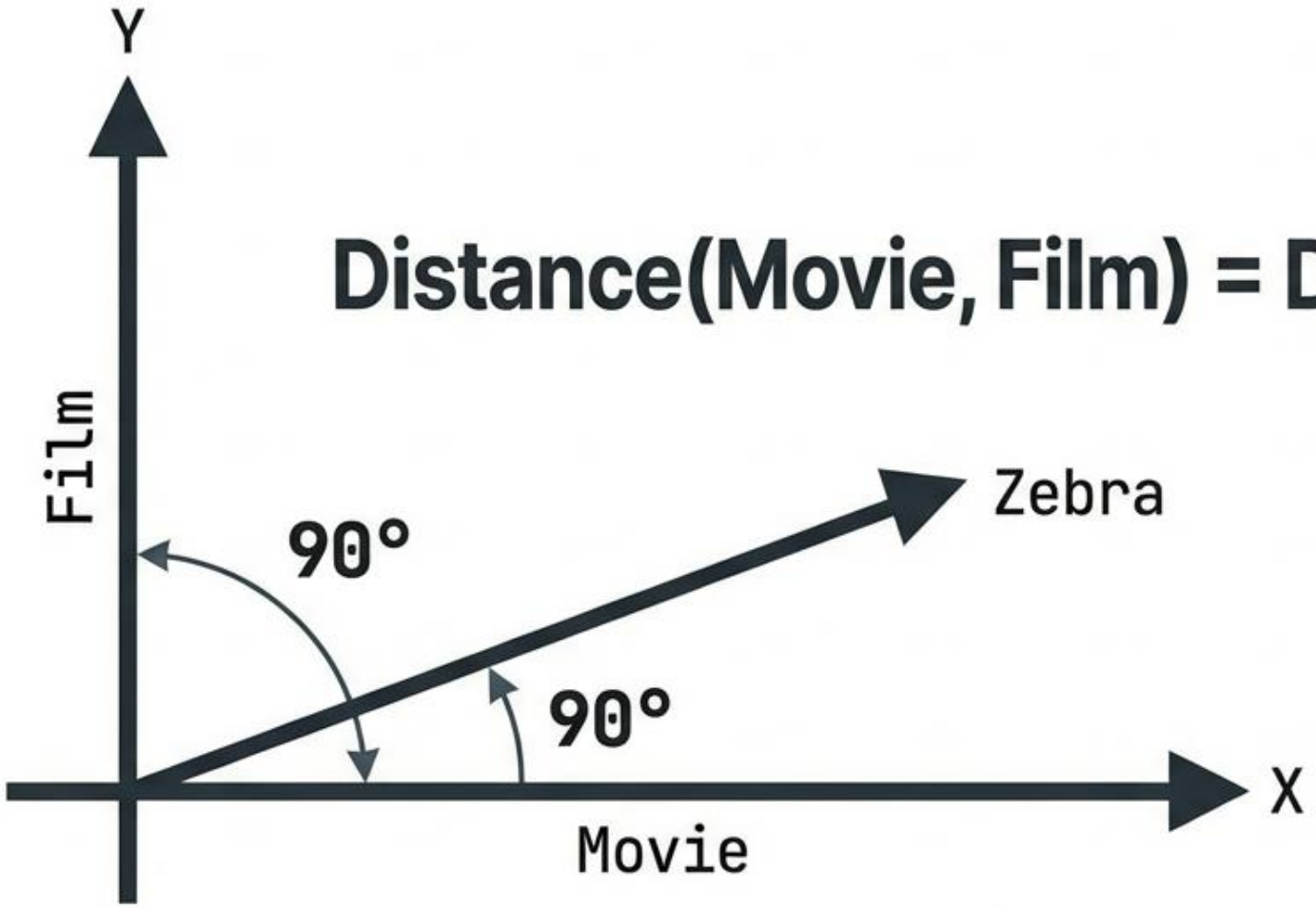
The Naive Approach: One-Hot Encoding

Vocabulary Size (N = 50,000)

the	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
cat	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
sat	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
on	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
mat	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Inefficient and Sparse. Every word is a vector with 49,999 zeros and a single 1.

The Orthogonality Problem: Inability to Capture Meaning



Distance(Movie, Film) = Distance(Movie, Zebra)

Verdict

Failure. The model cannot learn synonyms because mathematically, every word is equidistant from every other word.

Enter Word Embeddings: Dense, Learned, Meaningful

Before vs. After

One-Hot

Dimensions: 50,000

0
0
0
0
0
0
1
0
0
0
0
0
0
0
0
0
0
0

50,000

We compress the information into a smaller, continuous space

→

where every number captures a latent attribute.

Dense Embedding

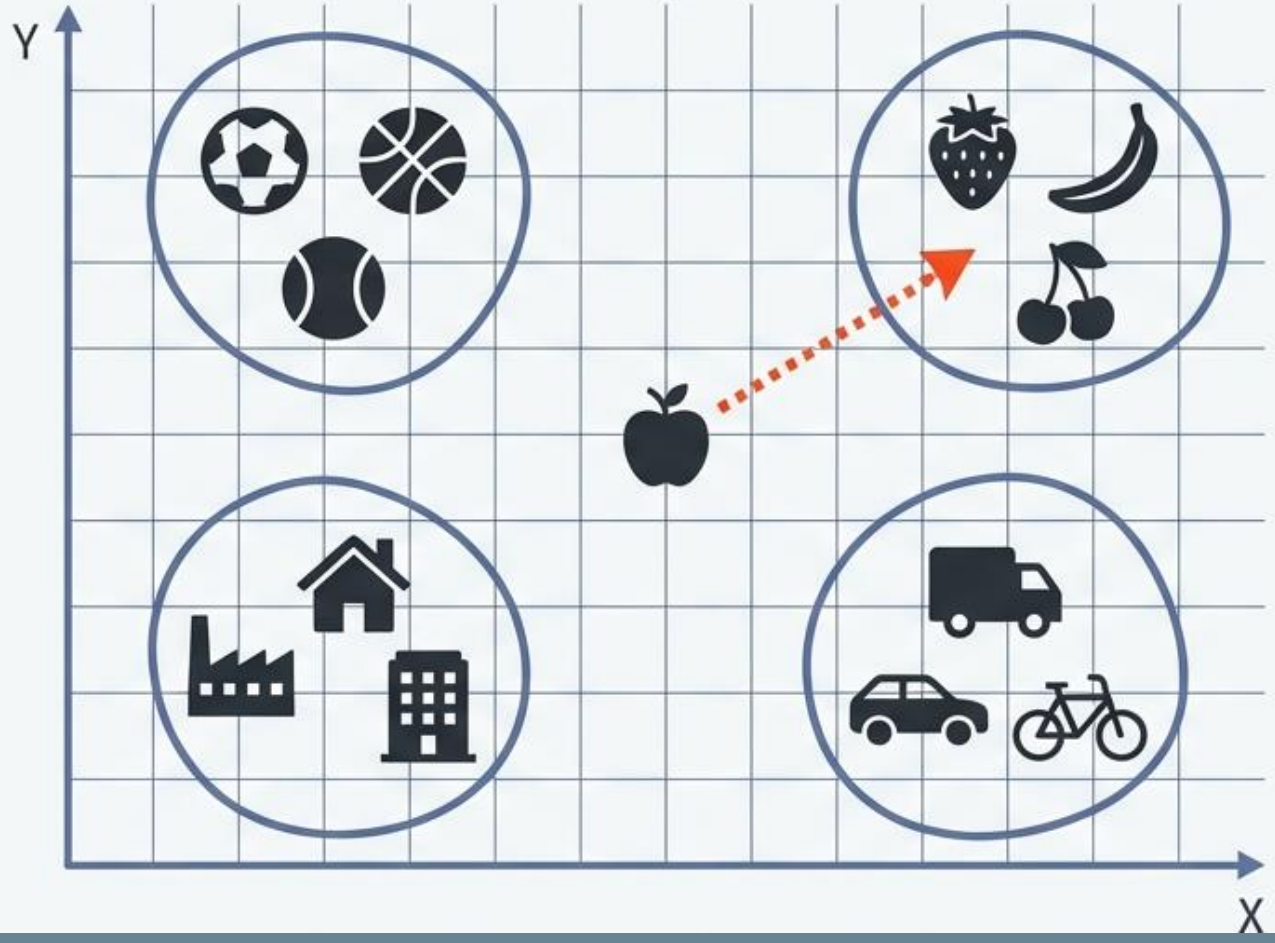
0.215	-0.489	0.801	0.003
-0.672	0.998	-0.124	0.555
0.998	0.124	0.555	-0.333
0.108	0.003	0.982	-0.582
0.486	0.581	0.274	0.004
0.823	0.646	0.575	-0.388
0.874	0.135	0.923	-0.333
0.003	0.124	-0.375	0.291

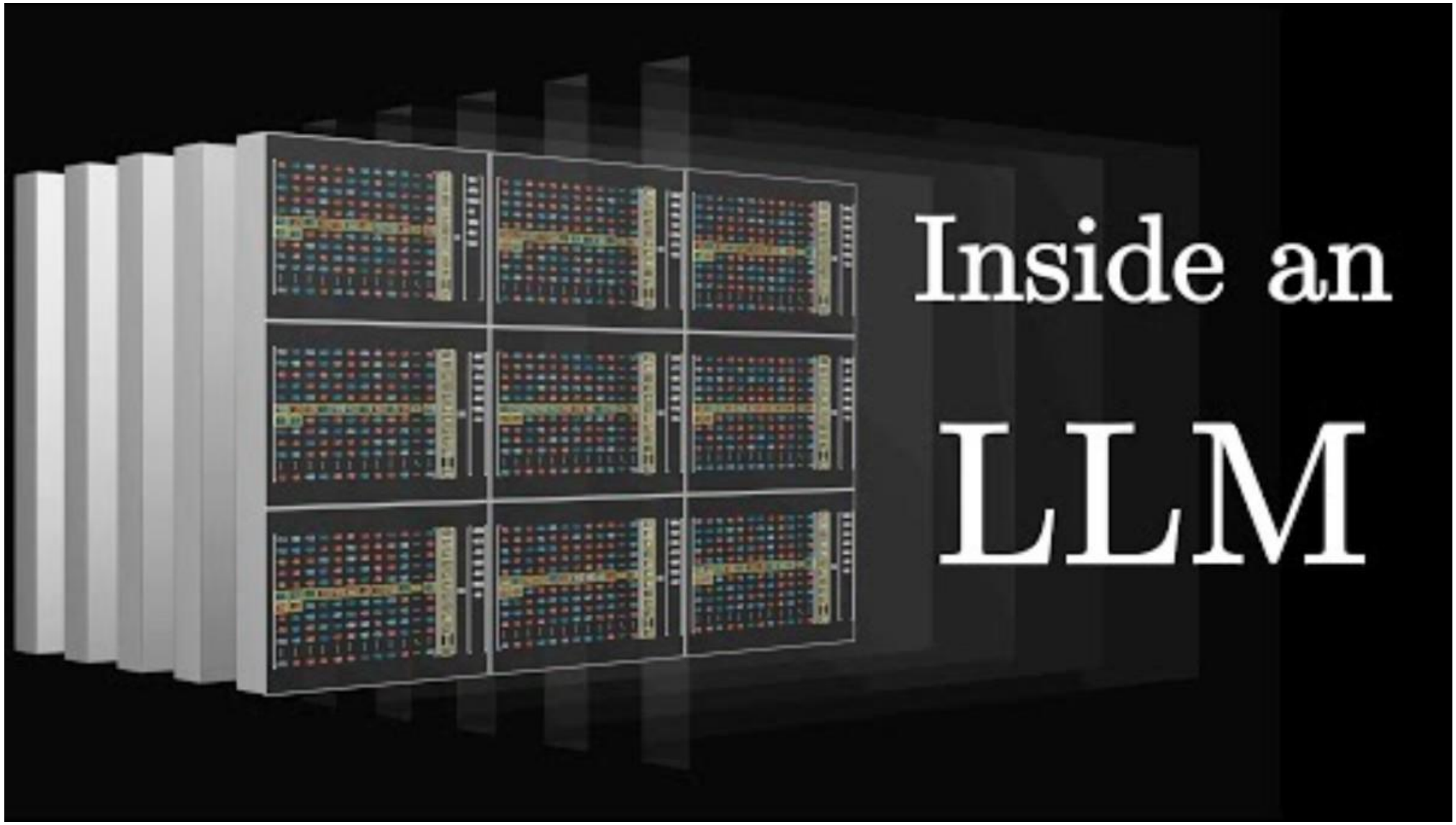
1024
Dimensions: 1024

Continuous vector space representation captures semantic relationships and similarities between words.

The Geometry of Meaning

In embedding space, synonyms are neighbors. The model learns to group concepts without explicit rules.




Inside an
LLM

Embedding: Then vs Now

Classically in NLP we would have a separate model which we trained to create embedding vectors. It acted like a lookup table.

- We learned *one vector per word* (e.g., word2vec / GloVe).
- Each word had *one fixed meaning*: *"bank" (river) = "bank" (money)*.
- We then fed those vectors into a simpler model.
- Often *frozen* during training: the downstream model learned on top of them.

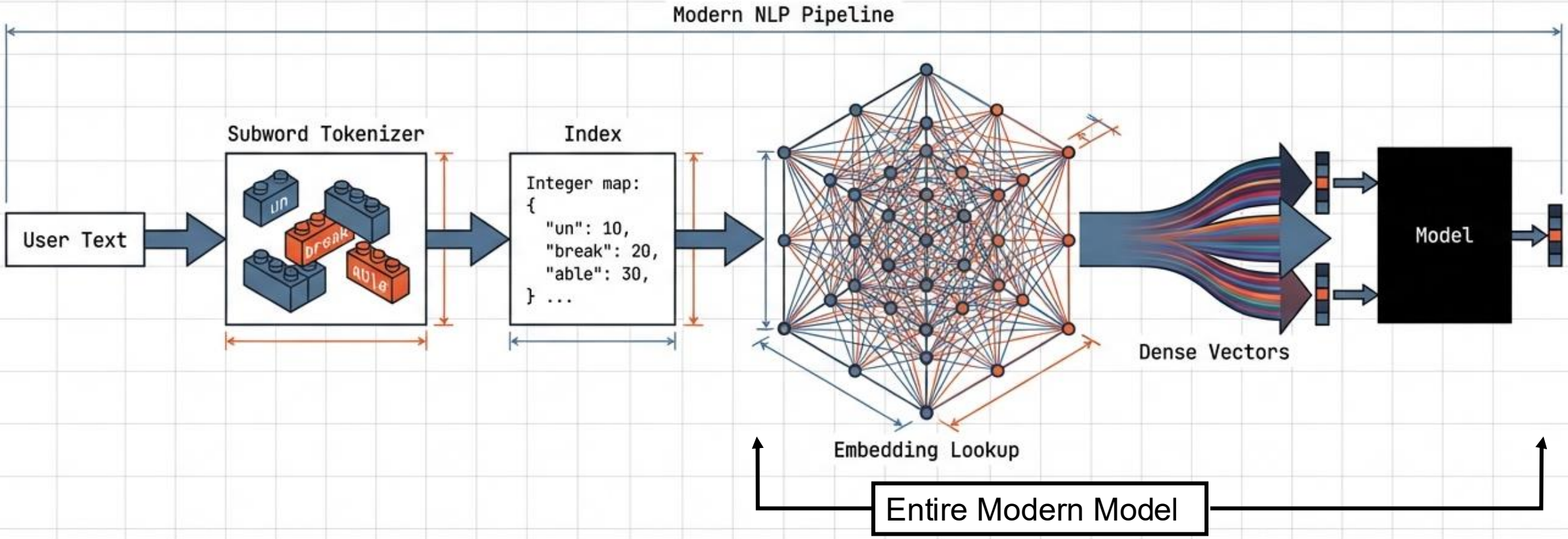
Embedding: Then vs Now

The modern approach (in LLMs) is learn the embeddings as apart of the model.

- Start with random embedding matrix and update with training.
- Tokens become vectors via this embedding matrix.
- These vectors are then subsequently transformed through the model into contextual embeddings
  - “bank” changes depending on the surrounding words.
- Often *outputs of models use same embedding matrix*.

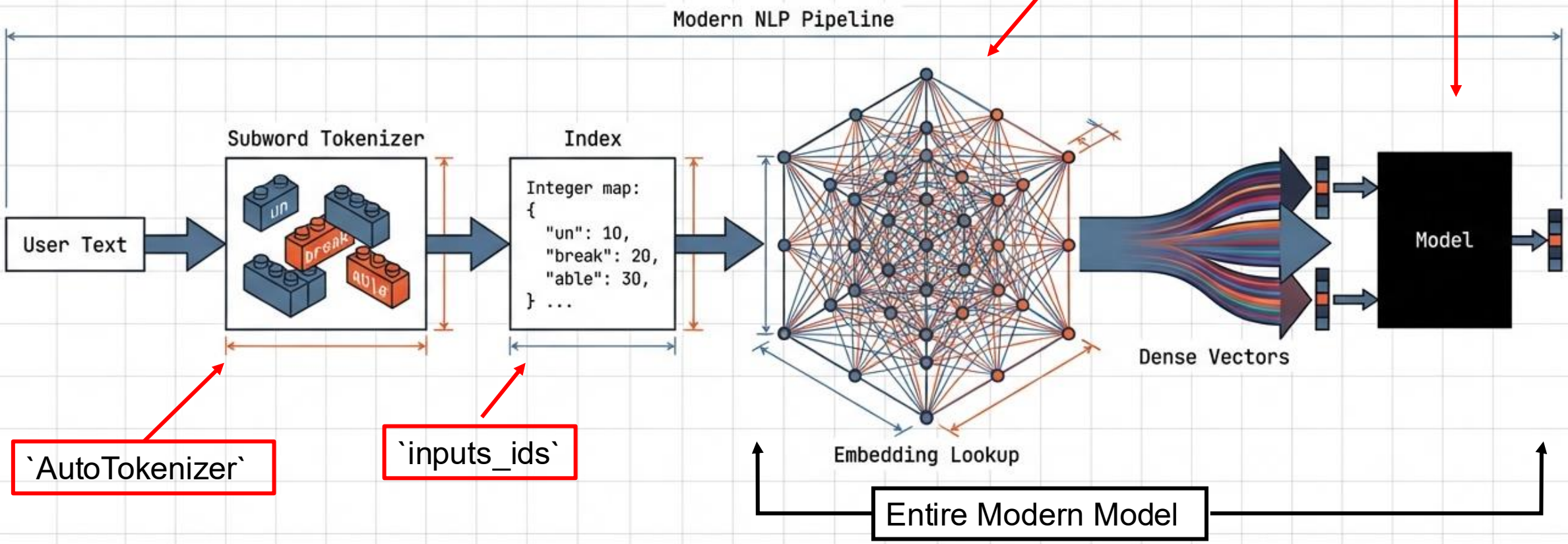
The Complete Picture

We have successfully translated language into a dense, structured numeric format.



The Complete Picture

We have successfully translated language into a dense, structured numeric format.



Exercise – Theory to Hugging Face